ClaimSense Command Reference

Technical Report

Document	CS-EVAL-CR-001
Issue	1
Date	14 November 2026
Owner	Eval & Red-Team
Status	For team use
Writing standard	ASD-STE100 Simplified Technical English, about 80% (Appendix C)
Source file	`docs/COMMAND_REFERENCE.md`. Edit this file only (section 18)

1. About this document

1.1 Purpose

This document gives the terminal commands for the ClaimSense project. Use it to install the project, examine the code, run the system and run each evaluation.

Each command has a purpose and a cost. The commands and their options come from the help text of each tool (`--help`) and from the project guides, on 14 November 2026.

1.2 Scope

This document covers these tasks:

- Installation, tests, lint and type checks.
- One claim, the case API, the case UI and Docker.
- The evaluation harness, the agent suite, the release gate and the ship check.
- Tool use, security, retrieval, judge calibration, experiments and Reflexion.
- Traces, observability, DeepEval and live monitoring.
- CI on GitHub, the documents and the course kit.

This document does not tell you how the code works. For that, read `docs/TEAM_GUIDE.md`. For the measured results, read the reports in `docs/`.

1.3 How to read a command

- Run each command from the root of the repository. A section tells you if a different folder is necessary.
- Activate the virtual environment before you run a command (section 2.2).
- The commands are for PowerShell on Windows. In Git Bash, change `$env:NAME = "value"` to `export NAME="value"`. All other commands are the same in the two shells.
- Replace a word in angle brackets with your value. For example, replace `<label>` with `ss-1114-check`.
- Keep `-X utf8` where a command shows it. It lets Python write Unicode text to a Windows console.
- The column "Cost" tells you if a command calls a paid model. "Free" means no model call.

- In the PDF, a long command can continue on a second line. Type it as one line, or copy it from the Markdown file.

1.4 Rules for every command

CAUTION: Do not print, copy or commit a key. The OpenRouter key comes from the environment variable `OPENROUTER_API_KEY`. Never commit the file `.env`.

CAUTION: A terminal in VS Code can contain the key, because VS Code loads `.env`. Then a command without `--stub` can call paid models. For a free run, set `$env:CLAIMSENSE_LLM = "stub"` first, or use `--stub`.

CAUTION: Do not read, copy or run the held-out set (`data/claimsense-heldout/`). Only one person from Eval & Red-Team runs it, one time (section 16).

CAUTION: Use a new label for each run. Some tools write over a run with the same label. Never use again a label that a report quotes, for example `drill-2`, `drill-3`, `real-drill` or `real-dev`.

- Use synthetic claims only. Do not put real health data in the project.
- Do not make a threshold easier to get a pass. Find the cause, or record the decision in `docs/adr/`.
- Edit Markdown files only. Then build the PDF, DOCX and HTML files again (section 18). Do not edit a generated file by hand.

1.5 Terms

Term	Meaning in this document
Adapter	The software that gives a claim to the thing under test and returns its answer
Checkpoint file	The SQLite file in which the case API keeps paused cases
Composite	The weighted score of a run, from 0.0 to 1.0. The pass mark is 0.85
Gate	A pass or fail test. One failed gate fails the whole run
Label	The name of a run. It names the output files in `runs/`
Real mode	The agents call paid models through OpenRouter
Release gate	The harness and the agent suite in one run (`eval.full`)
Ship check	Every evaluation stage in one command, with one verdict (`eval.ship`)
Span	One timed step in a trace
Stub mode	The agents use fixed rules and call no model. It is free, and each run gives the same answer

1.6 Abbreviations

Abbreviation	Meaning
API	Application Programming Interface
CI	Continuous Integration
KB	Knowledge Base
MCP	Model Context Protocol
OTel	OpenTelemetry
OTLP	OpenTelemetry Protocol
RAGAS	Retrieval Augmented Generation Assessment
STE	Simplified Technical English
UI	User Interface

---

2. Prepare the computer (one time)

2.1 Requirements

Item	Necessary for
Python 3.11 or later (CI uses 3.12)	All commands
Git	The repository
Microsoft Edge or Google Chrome	PDF builds (section 18)
Docker Desktop	Docker only (section 7). It is optional
An OpenRouter key	Paid commands only

2.2 Install the project

Do these steps one time on each computer. `requirements.txt` installs the project and every extra.

```
# 1. Get the repository
git clone https://github.com/Sanjaymartolia/ClaimSense-Insurance.git
cd ClaimSense-Insurance
# 2. Make and activate the virtual environment
python -m venv .venv
.\.venv\Scripts\Activate.ps1          # Git Bash: source .venv/Scripts/activate
# 3. Install everything
pip install -r requirements.txt
```

To install only the parts for your work, use the extras. For example:

```
pip install -e ".[dev,agents,ui]"
```

Extra	What it adds
`dev`	pytest, ruff, mypy, pypdf
`agents`	LangGraph and LangChain: the agents and the case API
`mcp`	FastMCP: the tool servers over MCP
`ui`	Streamlit: the case UI
`retrieval`	Chroma, BM25 and the reranker: the policy search
`eval`	RAGAS: the paid judge
`observability`	OpenTelemetry: traces
`deepeval`	DeepEval
`docs`	python-docx and matplotlib: DOCX files and charts
`course`	The course kit
`reflexion`	DSPy: the Reflexion experiments

2.3 Prepare the key file (paid runs only)

```
Copy-Item .env.example .env       # then put your key in .env with a text editor
[bool]$env:OPENROUTER_API_KEY     # True if this terminal has the key. It never shows the key
```

If the check shows `False` in a terminal outside VS Code, load the key from `.env`. These two lines do not show the key:

```
$line = Select-String -Path .env -Pattern '^\s*OPENROUTER_API_KEY\s*=' | Select-Object -First 1
$env:OPENROUTER_API_KEY = ($line.Line -split '=', 2)[1].Trim().Trim('"')
```

2.4 Make sure that the installation works

Task	Command	Cost
Run the product tests (about 5 min)	`pytest`	Free
Run the evaluation self-check (seconds)	`python -m eval.harness.suite`	Free

---

3. Examine the code before a commit

Task	Command	Cost
Run the product tests	`pytest`	Free
Run one test file	`pytest tests/calc/test_adjudication.py`	Free
Run one test	`pytest tests/calc/test_adjudication.py -k <name>`	Free
Find lint errors	`ruff check .`	Free
Repair the lint errors that ruff can repair	`ruff check --fix .`	Free
Format the code	`ruff format .`	Free
Make sure that the format is correct (as CI does)	`ruff format --check .`	Free
Do the type check	`mypy`	Free
Run the case API tests	`pytest tests/api`	Free
Run the case UI tests	`pytest tests/ui`	Free
Make sure that no tool definition changed	`python -m claimsense.hooks.pins`	Free
Record the tool definitions after a reviewed change	`python -m claimsense.hooks.pins --update`	Free
Run the evaluation tests (318 tests)	`pytest eval/tests -q`	Free
Run the evaluation package tests	`pytest eval/agentic/tests eval/ship/tests eval/experiments/tests eval/calibration/tests`	Free
Run the tests of the four MCP servers (about 2 min)	`$env:CLAIMSENSE_MCP_E2E = "1"; pytest tests/orchestration/test_mcp_services.py`	Free

CAUTION: `pytest eval/tests` writes over the tracked file `runs/suite-test.json`. After the tests, run `git checkout -- runs/suite-test.json`, unless you want to update it.

The evaluation tests need all the extras. With fewer extras, fewer tests are collected (297, not 318), and the test count check fails.

3.1 Run the CI checks on your computer

These commands do the same checks as the CI jobs `test` and `eval-gate` (section 17).

```
$env:CLAIMSENSE_LLM = "stub"
# job test
ruff check .
ruff format --check .
mypy
pytest
$env:LANGGRAPH_STRICT_MSGPACK = "true"; pytest tests/api; Remove-Item Env:LANGGRAPH_STRICT_MSGPACK
# job eval-gate
python -m claimsense.hooks.pins
python -m eval.harness.suite
python -m eval.harness.run --adapter claimsense-real-calc --split dev,redteam --label <label>
python -m eval.full --stub --label <label> --quiet
python -m eval.agentic.run --label <label>
pytest eval/agentic/tests eval/ship/tests eval/experiments/tests eval/calibration/tests
```

4. Run one claim and see its trace

Task	Command	Cost
Show the claim ids	`python scripts/run_one.py --list`	Free
Run one claim with real models and print each step	`python scripts/run_one.py CLM-0004`	Paid
Decide the review yourself at the terminal	`python scripts/run_one.py CLM-0004 --reviewer console`	Paid
Use the real tools, not the stubs	`python scripts/run_one.py CLM-0004 --real-tools`	Paid
Run the three demo scenes, stub mode	`python -X utf8 scripts/demo.py`	Free
Run one demo scene	`python -X utf8 scripts/demo.py --scene 3`	Free
Run the demo with real models and real tools	`python -X utf8 scripts/demo.py --llm real --tools real --pause`	Paid
Run the demo against a running case API	`python -X utf8 scripts/demo.py --url http://127.0.0.1:8000 --pause`	Free with a stub API
Print the full trace of a claim (agent suite)	`python -m eval.agent_suite --stub --label <label> --show CLM-0004`	Free
Print the OpenTelemetry span tree of a claim	`python -m eval.observability show --stub CLM-0004`	Free

`scripts/run_one.py` does not run in stub mode. `scripts/demo.py` asks before a paid model call, unless you add `--yes`. For the demo in detail, read `docs/DEMO_SCRIPT.md`.

5. The case API

The case API starts a claim, sends each step of the run as a stream, and takes the decision of the reviewer. For all its routes, read `src/claimsense/api/README.md`.

5.1 Start the API with stub agents

```
$env:CLAIMSENSE_LLM = "stub"
$env:CLAIMSENSE_REVIEWERS = "local-alice=alice:adjudicator,local-bob=bob:senior,local-alice-
senior=alice:senior"
$env:CLAIMSENSE_CHECKPOINTS = "claimsense_checkpoints.sqlite"
uvicorn claimsense.api.main:app --port 8000
```

5.2 Start the API with real models

CAUTION: Each claim costs about $0.0013 with the current models. Make sure that the key check shows `True` before you start the API. Without a key, the API starts, but the first claim fails.

```
$env:CLAIMSENSE_LLM = "real"
$env:CLAIMSENSE_REVIEWERS = "local-alice=alice:adjudicator,local-bob=bob:senior,local-alice-
senior=alice:senior"
$env:CLAIMSENSE_CHECKPOINTS = "claimsense_checkpoints_real.sqlite"
[bool]$env:OPENROUTER_API_KEY
uvicorn claimsense.api.main:app --port 8000
```

Use a different checkpoint file for each mode. The API keeps each case under its case id. Thus a case that you ran in stub mode is refused in real mode: "the case exists already".

To change the mode, stop the API (Ctrl+C), set the variables again, and start the API again.

5.3 Settings of the API

Variable	Values	Effect
`CLAIMSENSE_REVIEWERS`	`<token>=<id>:<level>` , with commas between reviewers	The reviewer tokens. Without it, the case routes answer 503
`CLAIMSENSE_LLM`	`stub` or `real`	Fixed rules, or paid models
`CLAIMSENSE_TOOLS`	`stub` , `real` or `mcp`	Which tools the agents use (section 5.5)
`CLAIMSENSE_CHECKPOINTS`	A file name, or `:memory:`	Where the paused cases stay
`CLAIMSENSE_AUDIT_LOG`	A file name	The audit trail, one JSON line for each event
`CLAIMSENSE_OTEL`	`console` or `otlp`	One trace for each claim run. Needs the `observability` extra
`CLAIMSENSE_APPROVAL_SECRET`	A long random text	Signs the approvals. Necessary for real tools

5.4 Examine a running API

Task	Command or address
Is the API up, and in which mode?	`Invoke-RestMethod http://127.0.0.1:8000/health`
Interactive documentation	Open `http://127.0.0.1:8000/docs` in a browser
Metrics for Prometheus	`Invoke-RestMethod http://127.0.0.1:8000/metrics/`
Show the demo claims	`Invoke-RestMethod http://127.0.0.1:8000/demo/claims -Headers @{Authorization = "Bearer local-alice"}`
Show one case	`Invoke-RestMethod http://127.0.0.1:8000/claims/CLM-0004 -Headers @{Authorization = "Bearer local-alice"}`

5.5 Real tools and tools over MCP

`real` tools need three things. Do these steps one time:

```
python -m claimsense.mcp_servers.seed_dev_policies
python -m claimsense.retrieval.index
$env:CLAIMSENSE_TOOLS = "real"
$env:CLAIMSENSE_APPROVAL_SECRET = "<a long random text>"
```

For `mcp` tools, also install the `mcp` extra. The API starts the four tool servers when it starts. The first start can take two minutes. The API does not start if a server advertises tools that are different from `config/tool_pins.json`.

---

6. The case UI

The case UI is the screen of the adjudicator. It is a client of the case API only.

```
# terminal 1: start the case API (section 5.1 or 5.2)
# terminal 2: start the UI, then open http://127.0.0.1:8501
python -m claimsense.ui
```

Task	Command
Use a different API address	`$env:CLAIMSENSE_API_URL = "http://<host>:<port>"` before you start the UI
Use a different port	`python -m claimsense.ui --server.port 8600`
Run the UI tests	`pytest tests/ui`

To do the demo, follow these steps in the UI:

1. Sign in two times in the sidebar: with `local-alice` (adjudicator) and with `local-bob` (senior).
2. Use "Acting as" to change the reviewer.
3. Click the demo scenes in sequence: 1, 2, the same bill again, then 3.
4. For each review, select Approve, Edit a line, or Reject.
5. Read the caption below the title. It shows the mode of the API: `agents: stub` or `agents: real`.

On a claim that is not admissible (for example CLM-0001), Approve means repudiate. An edit on that claim overrides the finding. Then coverage, the fraud check and the calculator run again. For the full demo, read `src/claimsense/ui/README.md`.

---

7. Docker

Docker is optional. Sections 5 and 6 run the same API and UI without Docker. The CI job `compose` builds and tests both images after each push.

7.1 Install Docker Desktop

```
winget install -e --id Docker.DockerDesktop
# Start Docker Desktop one time. It can ask for WSL 2 and a restart.
# Then open a new terminal:
docker --version
docker compose version
```

7.2 Start the API and the UI

CAUTION: Run one of the two blocks, not the two blocks. `docker compose up` continues until you push Ctrl+C.

```
# stub agents (free)
$env:CLAIMSENSE_LLM = "stub"
$env:CLAIMSENSE_REVIEWERS = "local-alice=alice:adjudicator,local-bob=bob:senior,local-alice-
senior=alice:senior"
docker compose up --build api ui

# real models (paid): the key goes to the API container from .env
$env:CLAIMSENSE_LLM = "real"
$env:CLAIMSENSE_REVIEWERS = "local-alice=alice:adjudicator,local-bob=bob:senior,local-alice-
senior=alice:senior"
docker compose up --build api ui
```

`$env:CLAIMSENSE_LLM` stays set in the terminal until you change it or close the terminal. Other commands in that terminal also read it.

7.3 Start the full stack

```
docker compose up --build
```

Service	Address
Case API	`http://127.0.0.1:8000`
Case UI	`http://127.0.0.1:8501`
Chroma (vector store)	`http://127.0.0.1:8001`
Prometheus	`http://127.0.0.1:9090`
Grafana	`http://127.0.0.1:3000`

7.4 Stop, logs and old cases

Task	Command
Stop the containers	`docker compose down`
Show the API log	`docker compose logs api`
Find the volume that keeps the cases	`docker volume ls`
Remove the old cases only (after `docker compose down`)	`docker volume rm <name ending in _case-data>`

8. The evaluation harness: is the final answer correct?

Task	Command	Cost
Run the whole suite and compare it with the recorded outcomes	`python -m eval.harness.suite`	Free
The ceiling: replay the gold (the score must be about 1.0)	`python -m eval.harness.run --adapter gold-replay --split dev --label <label>`	Free
The floor: an agent that does nothing	`python -m eval.harness.run --adapter null --split dev --label <label>`	Free
Score the real pipeline with stub agents	`$env:CLAIMSENSE_LLM = "stub"; python -m eval.harness.run --adapter claimsense --split dev,redteam --label <label>`	Free
Score the real pipeline with real models	`python -m eval.harness.run --adapter claimsense --split dev,redteam --label <label>`	Paid
Use the product calculator (the CI gate)	`python -m eval.harness.run --adapter claimsense-real-calc --split dev,redteam --label <label>`	Free in stub mode
Use the hybrid retriever (needs the index)	`python -m eval.harness.run --adapter claimsense-real-retrieval --split dev --label <label>`	Free in stub mode
Prove that a gate fires: inject a fault	`python -m eval.harness.run --adapter perturbed --fault unsafe_settle --split dev --label <label>`	Free
Score the rupee amounts strictly	add `--tolerance-inr 0`	Free
Add the RAGAS judge for five claims	add `--judge --judge-limit 5`	Paid
Accept a changed composite of the pipeline	`python -m eval.harness.suite --accept-pipeline`	Free

The faults are `unsafe_settle`, `obey_injection`, `skip_calculator`, `drop_flags`, `fake_citation` and `shift_amounts`. A run with a fault must fail with exit code 1.

The suite always uses stub agents. It writes over the tracked file `runs/suite.json`.

After `--accept-pipeline`, record the new value in `eval/harness/suite.py` (`ROWS`), in `eval/README.md` and in the report. Do not change the expected result of a gate row.

The RAGAS judge failed its calibration on 14 November 2026. Its weight is 0.00. Do not put its score in a report.

9. The release gate and the agent suite

Task	Command	Cost
Release gate: harness and agent suite, stub (about 1 min)	`python -m eval.full --stub --label <label>`	Free
Release gate with real models, as CI does (a few minutes)	`python -m eval.full --label <label> --splits dev,redteam --workers 4`	Paid, about $0.07
Release gate with the hybrid retriever	add `--variant real-retrieval`	as above
Print the full trace of some claims	add `--show CLM-0004,CLM-9001`	as above
Agent suite: what did the agents do?	`python -m eval.agent_suite --stub --label <label>`	Free
Compare two variants	`python -m eval.agent_suite --stub --label <label> --variant v1,guarded`	Free
Measure pass@k (three runs for each case)	add `--k 3`	as above
Add the payload judge	add `--judge`	Paid

The variants are `v1`, `guarded`, `lean-pr`, `no-docs` and `real-retrieval`.

`eval.full` and `eval.agent_suite` refuse a stub run if its label holds a real-model run. Use a new label.

10. The ship check: is this commit safe to ship?

The ship check runs each evaluation stage as a separate process and gives one verdict.

Task	Command	Cost
Rehearsal: all stages, stub agents (about 15 min)	`python -X utf8 -m eval.ship --label <label> --stub`	Free
Show the stages	`python -X utf8 -m eval.ship --label <label> --list`	Free
Leave out the slow retrieval stage	add `--skip retrieval`	as above
Run some stages only	add `--only release-gate,tool-use`	as above
The ship decision, with real models	`python -X utf8 -m eval.ship --label <label>`	Paid
Add the adaptive attacker	add `--attack`	Paid

The stages are `tests`, `lint`, `format`, `tool-pins`, `release-gate`, `tool-use`, `deepeval`, `deepeval-judge`, `security`, `retrieval` and `ragas`. The first seven can block the verdict.

The results are in `runs/ship/<label>/summary.md`. The log of each stage is in `runs/ship/<label>/logs/`. For the details, read `docs/SHIP_CHECK.md`.

11. Tool use, security, retrieval and the data sets

Task	Command	Cost
Tool use: tool calls, hooks, guardrails and database state	`python -m eval.agentic.run --label <label>`	Free
Security scoreboard: each defence against each attack	`python -m eval.security scoreboard --stub --label <label>`	Free
Security scoreboard with real models	`python -m eval.security scoreboard --label <label>`	Paid
Adaptive attacker (exit code 1 means that the attacker won)	`python -m eval.security attack --label <label>`	Paid
Build the policy search index	`python -m claimsense.retrieval.index`	Free
Measure the search on 47 questions (about 10 min)	`python -m claimsense.retrieval.evaluate --label <label>`	Free
Extract the policy clauses again	`python -m claimsense.ingestion.extract_policy_clauses`	Free
Build the knowledge base again	`python -m claimsense.ingestion.build_policy_knowledge_base`	Free
Build the red-team set again	`python eval/redteam/build_cases.py`	Free

The search chart goes to `runs/retrieval_<label>.png`. It needs the `retrieval` and `docs` extras.

Do not edit `data/knowledge_base/`, `data/processed/` or the red-team set by hand. Build them again with the commands above. Build the red-team set again after each change to the dev gold.

---

12. Judge calibration

The judge must agree with people before its score counts. The procedure is in `eval/calibration/README.md`.

Task	Command	Cost
Calculate the two kappas again from the saved scores	see the block below	Free
Collect new items from the real agents	`python -X utf8 -m eval.calibration.collect collect`	Paid, small
Make one sheet for each rater	`python -X utf8 -m eval.calibration.collect sheets`	Free
Import the labels of the two raters	`python -X utf8 -m eval.calibration.collect import --a eval/calibration/sheet_rater_a.csv --b eval/calibration/sheet_rater_b.csv`	Free
Score with the RAGAS judge	`python -X utf8 -m eval.calibration.collect score`	Paid, small
Score with the rubric judge (the rule of the raters)	`python -X utf8 -m eval.calibration.collect score --judge rubric`	Paid, about $0.01
Run the calibration tests	`pytest eval/calibration/tests`	Free

```
python -X utf8 -c "import json; from eval.calibration.judge_calibration import *; s =
json.load(open('eval/calibration/judge_scores.json'))['scores'];
print(report(load_calibration_set(), s))"
```

Round 2 uses new items in a new folder. Do these steps in sequence:

```
$dir = "eval/calibration/round2"          # Git Bash: dir="eval/calibration/round2"
# 1. Collect new items. Skip the items of round 1
python -X utf8 -m eval.calibration.collect collect --real-retrieval --out
"$dir/calibration_set.jsonl" --exclude eval/calibration/calibration_set.jsonl
# 2. Make the sheets. Two raters label them independently
python -X utf8 -m eval.calibration.collect sheets --out "$dir/calibration_set.jsonl"
# 3. Import the labels
python -X utf8 -m eval.calibration.collect import --a "$dir/sheet_rater_a.csv" --b
"$dir/sheet_rater_b.csv" --out "$dir/calibration_set.jsonl"
# 4. Score the two judges on the same items
python -X utf8 -m eval.calibration.collect score --judge rubric --out "$dir/calibration_set.jsonl"
python -X utf8 -m eval.calibration.collect score --out "$dir/calibration_set.jsonl"
```

`collect` refuses a file that exists, and it refuses stub mode. `score` stops before the paid judge if the raters do not agree. It writes the scores next to the set, one file for each judge.

---

13. Experiments and Reflexion

Task	Command	Cost
Experiment E1 (also E2 or E4), plumbing check	`python -X utf8 -m eval.experiments.run --experiment E1 --label <label> --stub`	Free
Experiment E1 with real models	`python -X utf8 -m eval.experiments.run --experiment E1 --label <label>`	Paid
Run one configuration	`python -X utf8 -m eval.experiments.run --config <file.yaml> --label <label>`	Paid in real mode
E3: learn lessons; a person approves each lesson	`python -X utf8 -m eval.experiments.e3 learn --reviewer "<name>" --out <folder>`	Paid in real mode
Run the experiment tests	`pytest eval/experiments/tests`	Free

The Agents & Orchestration owner keeps the Reflexion code. Do not change it without the owner. These are its commands:

Task	Command	Cost
Baseline with an empty playbook	`python scripts/run_reflexion_experiment.py baseline --out <folder>`	Paid in real mode
Full experiment; a person approves each lesson	`python scripts/run_reflexion_experiment.py experiment --reviewer "<name>" --out <folder> --after-runs 2`	Paid in real mode
DSPy and GEPA, harness check only	`python scripts/run_reflexion_dspy.py --out <folder> --tolerance-rs 1 --gepa-metric-calls 20`	Free in stub mode
DSPy and GEPA, real run	`python scripts/run_reflexion_dspy.py --out <folder> --tolerance-rs 1`	Paid, cents

The output of `--approve auto` is a harness check only. Never use it as a result.

---

14. Traces, observability and DeepEval

14.1 Traces

Task	Command	Cost
Trace claims in memory and print the span tree	`python -m eval.observability show --stub CLM-0004`	Free
Also print two spans as OTLP JSON	add `--json`	Free
Download the Jaeger viewer (one time)	`python -m eval.observability setup`	Free
Normal traffic and a tool outage, sent to Jaeger	`python -m eval.observability run --stub --label <label> --open`	Free
Keep Jaeger open after the run	add `--keep`	Free
Send the spans to another backend (Langfuse, LangSmith)	add `--backend otlp-env`, and set the `OTEL_EXPORTER_OTLP_` variables	Free
Stop a Jaeger that an interrupted run left open	`python -m eval.observability down`	Free
Trace each run of the case API	`$env:CLAIMSENSE_OTEL = "console"` before you start the API	as the API
Write the audit trail of the case API	`$env:CLAIMSENSE_AUDIT_LOG = "audit.jsonl"` before you start the API	as the API

`eval.observability run` refuses a label that exists. Exit code 0 means that each check holds.

CAUTION: Do not set `LANGSMITH_TRACING=true`. LangSmith's own tracing can copy model text into an error. To send spans to LangSmith, use `--backend otlp-env`.

14.2 DeepEval

CAUTION: Set the two DeepEval variables before you run `deepeval`. If you do not, DeepEval loads the key from `.env`, and the tests can call paid models.

```
$env:DEEPEVAL_DISABLE_DOTENV = "1"
$env:DEEPEVAL_TELEMETRY_OPT_OUT = "1"
$env:CLAIMSENSE_LLM = "stub"
deepeval test run eval/deepeval_suite          # 74 tests, free
# Add the paid judge:
$env:CLAIMSENSE_DEEPEVAL_JUDGE = "1"
deepeval test run eval/deepeval_suite
```

DeepEval needs the `deepeval` extra. For the details, read `docs/OBSERVABILITY_REPORT.md`.

---

15. Live monitoring

Task	Command	Cost
Download Prometheus and Grafana (one time, about 1.5 GB)	`python -m eval.monitoring setup`	Free
Run the incident drill (about 6 min)	`python -m eval.monitoring run --stub --label <label> --open`	Free
Keep the dashboard open after the drill	add `--keep`	Free
Stop the servers that an interrupted run left open	`python -m eval.monitoring down`	Free
Write the alert rules and dashboards for Deploy & Ops	`python -m eval.monitoring export --out runs/monitoring/export`	Free

CAUTION: A label that exists is written over. Never use `drill-2`, `drill-3` or `real-drill` again. A drill with real models cost $0.17.

Only one drill can run at a time. Exit code 3 means that the result is not certain. Then run the drill again. Do not edit the folder `monitoring/`: it belongs to Deploy & Ops.

---

16. The held-out set (Eval & Red-Team only)

CAUTION: Do not read, copy, generate or change a held-out case. Do not put a gold answer into a prompt or a tool.

Only one person from Eval & Red-Team runs the held-out set. That person runs it one time, with the code, the model, the prompts and the playbook frozen. The results go to a folder outside the repository.

```
python -X utf8 -m eval.experiments.e3 heldout --heldout data/claimsense-heldout --out
../claimsense-heldout-results/<run>
```

The script stops in these conditions:

- The held-out folder is in a different place in the repository.
- The output folder is in the repository.
- The playbook is not frozen, or it changed after the freeze.
- A claim id is also in the dev set.

---

17. CI on GitHub

Event	Jobs	Cost
A push to `main`, or a pull request	`test`, then `compose` and `eval-gate`	Free
Actions, then ci, then Run workflow (manual)	the same jobs, then `eval-real`	Paid, about $0.07

Job	What it does
`test`	ruff, the format check, mypy, pytest, and the API tests with strict checkpoint loading
`compose`	Builds the API and UI images, starts them, and sends a demo claim
`eval-gate`	Tool pins, the harness suite, the eval gate, the release gate, tool use, and the package tests
`eval-real`	The release gate with real models. It needs the secret `OPENROUTER_API_KEY`

To start the real run, open the repository on GitHub. Click Actions, then ci, then Run workflow. Keep the branch `main`, and click Run workflow. The result of `eval-real` is the artifact `eval-real-<run id>` on the page of the run.

To do the same checks on your computer, use section 3.1. For the full guide, read `docs/CI_CD_PIPELINE.md`.

---

18. Documents

Task	Command
Build the team guide again (PDF, DOCX and HTML)	`python scripts/build_team_guide.py`
Build the harness report DOCX again	`python -m eval.tools.build_report_docx`
Make sure that the harness report is correct	`pytest eval/tests/test_report_doc.py -q`
Build another report DOCX again	`python -m eval.tools.build_report_docx --source docs/<file>.md --target docs/<file>.docx`
Build the technical workflow PDF again	`python scripts/build_pdf.py`
Show the framework rubric score	`python -m eval.tools.rubric_score`
Show what would change the rubric score	`python -m eval.tools.rubric_score --plan`

The PDF builds need Microsoft Edge or Google Chrome. The DOCX builds need the `docs` extra.

This document: the source is `docs/COMMAND_REFERENCE.md`. The PDF comes from it through the converter in `scripts/build_pdf.py`. Edit the Markdown file only. A one-command build for this document is an open item.

---

19. The course kit

Work in your own folder, `exercises/<name>/`. Do not change the folder `course/`.

Task	Command	Folder	Cost
Make your copy of the kit	`python course/scripts/new_student.py <name>`	repository root	Free
Run the kit tests (not in CI)	`pytest course/tests`	repository root	Free
Dev evaluation, with a chart	`python scripts/evaluate_dev.py --label <label>`	`exercises/<name>/`	Paid
Add the 30 red-team claims	`python scripts/evaluate_dev.py --label <label> --redteam`	`exercises/<name>/`	Paid
Search quality only	`python scripts/evaluate_dev.py --retrieval-only --label <label>`	`exercises/<name>/`	Free
Run one claim with its trace	`python scripts/run_one.py CLM-0004`	`exercises/<name>/`	Paid
Stop before settlement, then approve	`python scripts/run_one.py CLM-0004 --pause` , then `python scripts/approve.py CLM-0004 --yes`	`exercises/<name>/`	Paid
Build the kit search index again	`python scripts/build_index.py --force --strategy clauses`	`exercises/<name>/`	Free

Appendix A. Exit codes

Command	0	1	2	3
`eval.harness.run`	All gates hold, composite at or above the pass mark	A gate failed, or the composite is low	—	—
`eval.harness.suite`	Each row agrees with the recorded outcome	A row is different	—	—
`eval.full` , `eval.agent_suite`	The run passes	The run does not pass	—	—
`eval.ship`	SHIP (real) or REHEARSAL PASSED (stub)	DO NOT SHIP	NOT VERIFIED: a stage could not run	—
`eval.agentic.run`	No case failed without an expected reason	A case failed	—	—
`eval.security attack`	The attacker did not win	The attacker won	—	—
`eval.observability run`	Each check holds	A check failed	It cannot start	—
`eval.monitoring run`	The drill passed	The drill failed	It cannot start, or another run is active	The result is not certain
`eval.calibration.collect collect`	20 items or more	Fewer than 20 items	A file exists, or stub mode	—
`eval.calibration.collect score`	The judge ran	The raters do not agree	—	—

Appendix B. Quick reference card

```
# every day (free)
pytest
ruff check .
ruff format --check .
mypy
python -m eval.harness.suite
python -m eval.full --stub --label <label>
python -m eval.tools.rubric_score

# the case API and the case UI (two terminals; stub mode)
$env:CLAIMSENSE_LLM = "stub"
$env:CLAIMSENSE_REVIEWERS = "local-alice=alice:adjudicator,local-bob=bob:senior"
uvicorn claimsense.api.main:app --port 8000
python -m claimsense.ui

# before a release (free rehearsal, about 15 min)
python -X utf8 -m eval.ship --label <label> --stub

# traces (free)
python -m eval.observability show --stub CLM-0004
python -m eval.agent_suite --stub --label <label> --show CLM-0004

# paid: a key is necessary, and each run costs money
python scripts/run_one.py CLM-0004
python -m eval.full --label <label> --splits dev,redteam --workers 4
python -X utf8 -m eval.ship --label <label>
```

Appendix C. Compliance with ASD-STE100

This document follows ASD-STE100 Simplified Technical English for about 80% of its text.

Rule	How this document applies it
One instruction in each sentence	Each step and each table row gives one task
Procedure sentences have 20 words or fewer	Applied in the procedures and the cautions
Description sentences have 25 words or fewer	Applied in the descriptions
Use the imperative in procedures	"Run", "Set", "Use", "Do not"
Use the active voice	Applied, with some exceptions in the tables
A caution comes before the step	Each CAUTION is above its command
Use approved words	Applied where possible. Software verbs such as "run", "build" and "install" are kept
Keep technical names	Commands, options, file names and variable names are not changed

The exceptions are these: the technical names, some table cells that are noun phrases, and some long commands. These items are approximately 20% of the text.